

Kenbunshoku

Code Implementation Plan

Global AI Hackathon Series with Qwen Cloud — Track 5: EdgeAgent

1. Repository Structure

```
kenbunshoku/
├── camera-simulator/      # phone-as-IP-camera test harness (setup notes, not
app code)
├── edge-service/          # Dockerized local detection service
│   ├── detector.py        # YOLOv8n motion + person detection
│   ├── forwarder.py       # sends cropped frames to cloud ingestion API
│   ├── Dockerfile
│   └── requirements.txt
├── cloud-backend/         # deployed on Alibaba Cloud
│   ├── ingestion_api.py   # receives frames from edge-service
│   ├── qwen_client.py     # calls Qwen-VL via Qwen Cloud API
│   ├── memory_store.py    # pattern/visit history (SQLite or Redis)
│   ├── alert_dispatcher.py # decides + sends notification
│   └── deploy/            # Alibaba Cloud deployment config + proof-of-
deployment script
├── notification-client/   # Flutter app receiving push alerts
├── docs/                 # architecture diagram, write-up, video link
├── LICENSE               # MIT (public, visible in About section)
└── README.md
```

2. Component Breakdown

2.1 camera-simulator (test tool, not shipped)

Not production code — just the setup used to make a phone behave like a real IP camera for testing.

- Android: IP Webcam (free) — exposes an MJPEG/RTSP stream from the phone
- iOS: any free RTSP-broadcast app, or fallback to periodic still-frame capture over HTTP if RTSP isn't available
- Output: a stream URL the edge-service points to — identical in shape to what a real doorbell cam would expose

2.2 edge-service

A small Docker container, portable to any real edge device later (Raspberry Pi, NVR box, camera's own chip).

- detector.py — pulls frames from the camera stream, runs YOLOv8n for person/motion detection
- forwarder.py — on a positive detection, crops the relevant region and POSTs it to the cloud ingestion API (not full video, to stay bandwidth-aware)
- Runs locally on the M3 Max laptop for MVP testing; architecture doesn't assume this — any Docker host works

2.3 cloud-backend (Alibaba Cloud)

- `ingestion_api.py` — lightweight HTTP endpoint (FastAPI) that receives frames from any edge-service instance
- `qwen_client.py` — sends frames + a short context prompt to Qwen-VL via the OpenAI-compatible Qwen Cloud API
- `memory_store.py` — stores a rolling history of visits (timestamp, rough visual descriptor, classification) to detect recurring patterns
- `alert_dispatcher.py` — combines the Qwen classification with memory lookups, decides alert wording, sends the push notification
- `deploy/` — Alibaba Cloud deployment config (ECS or Function Compute) plus the script/recording used as proof-of-deployment for submission

2.4 notification-client

A minimal Flutter app (matching existing skillset) that receives push notifications and displays the agent's reasoning in plain language. No control actions exposed — display only, by design.

3. API Contract (edge → cloud)

POST `/ingest`

Content-Type: `multipart/form-data`

fields: `frame (image)`, `camera_id (string)`, `timestamp (ISO 8601)`

Response 200:

```
{ "classification": "familiar|delivery-like|anomalous",  
  "reasoning": "short plain-language explanation",  
  "alert_sent": true|false }
```

4. Data Flow (step by step)

1. Camera stream (phone or real device) is picked up by edge-service
2. `detector.py` flags a person/motion event
3. `forwarder.py` crops the frame and POSTs it to `/ingest` on the Alibaba Cloud backend
4. `ingestion_api.py` hands the frame to `qwen_client.py`
5. `qwen_client.py` calls Qwen-VL for a context classification + short reasoning string
6. `memory_store.py` is checked/updated for recurring-pattern context (e.g., "3rd visit this week, same time window")
7. `alert_dispatcher.py` composes the final alert and pushes it to notification-client
8. Homeowner reads the alert and decides on any next step — no automated action is taken

5. Build Order (maps to Project Plan timeline)

Day	Build target
Jul 15	Repo skeleton, Qwen Cloud account + voucher, Docker base for edge-service
Jul 16	<code>detector.py</code> working against the phone test stream (local detection only)

Jul 17	Alibaba Cloud backend stood up; ingestion_api.py + qwen_client.py; first end-to-end classification
Jul 18	memory_store.py + alert_dispatcher.py + notification-client receiving real alerts
Jul 19	Proof-of-deployment recording, architecture diagram finalized, README + submission text
Jul 20	Final run-through, record demo video, submit

6. Testing Plan

- Unit-level: detector.py against a handful of pre-recorded test clips (person present / absent / passing by)
- Integration: full loop using the phone-as-camera stream end to end, at least 3 distinct "visitor" scenarios
- Pattern memory: simulate a recurring visit (same rough time/appearance) at least twice to confirm the classification shifts from "unfamiliar" to "recognized pattern"
- Failure mode: disconnect cloud access mid-test to confirm the graceful local-only fallback alert fires

7. Alibaba Cloud Deployment & Proof

- Deploy cloud-backend on Alibaba Cloud (ECS instance or Function Compute) using the \$40 hackathon voucher / free trial credits
- Record a short screen capture (separate from the demo video) showing the backend running on Alibaba Cloud
- Link a code file in the repo that clearly shows Alibaba Cloud service/API usage, as required by the submission rules

8. Tooling — Free & Open Source Only

Tool	Purpose	Cost
YOLOv8n	Person/motion detection	Free, open weights
Docker	Portable edge-service packaging	Free
FastAPI	Cloud ingestion API	Free, open source
Qwen Cloud (Qwen-VL)	Vision-language reasoning	Free trial / \$40 hackathon voucher
Alibaba Cloud (ECS/Function Compute)	Required deployment target	Hackathon credits / free tier
SQLite or Redis	Memory/pattern store	Free, open source
Firebase Cloud Messaging or ntfy.sh	Push notifications	Free tier
Flutter	Notification client app	Free, open source
IP Webcam (Android) / RTSP app (iOS)	Phone-as-camera test stream	Free